

DocuMind AI

07 - Manual Técnico y de Administración

Guía técnica de instalación, configuración, arquitectura, operación y soporte

Proyecto	DocuMind AI - Sistema Inteligente de Gestión y Análisis Documental
Autor	Miguel Andres Barrios Rodriguez
Versión	1.0
Fecha	Septiembre de 2026

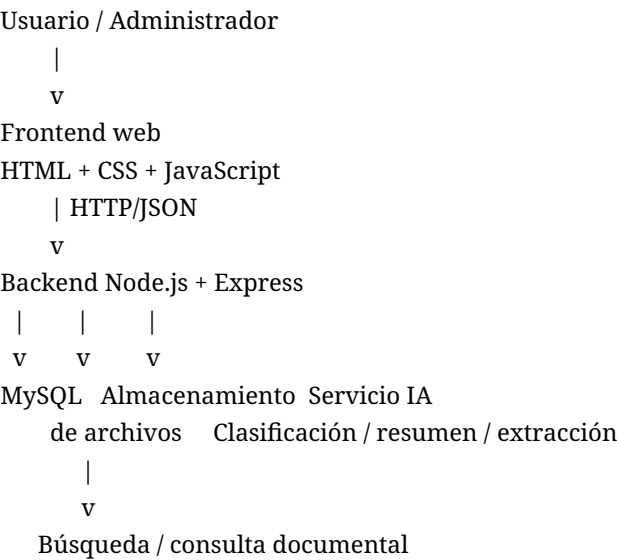
1. Introducción

Este manual describe los elementos técnicos necesarios para instalar, configurar, ejecutar, administrar y mantener DocuMind AI. Está dirigido a desarrolladores, administradores del sistema y personal de soporte que requiera reproducir el entorno o atender incidencias.

2. Objetivo

Proporcionar una guía reproducible de la arquitectura, dependencias, configuración, base de datos, almacenamiento, servicios de inteligencia artificial, seguridad, despliegue, respaldo y mantenimiento de la solución.

3. Arquitectura técnica



4. Tecnologías propuestas

Componente	Tecnología	Responsabilidad
Frontend	HTML, CSS, JavaScript	Interfaz de usuario, formularios, dashboard y consumo de API.
Backend	Node.js + Express	Autenticación, lógica de negocio, archivos, IA, búsqueda y API REST.
Base de datos	MySQL	Usuarios, repositorios, documentos, estados, resultados, consultas y errores.
Carga de archivos	Multer	Recepción y validación inicial de archivos.
PDF	pdf-parse u opción equivalente	Extracción de texto desde PDF.
DOCX	Mammoth	Extracción de texto desde documentos DOCX.
TXT	fs de Node.js	Lectura de texto plano.
Autenticación	JWT + bcrypt	Control básico de acceso y protección de contraseñas.
IA	API seleccionada, por ejemplo Gemini u OpenAI	Clasificación, resumen, extracción y respuestas.
Configuración	dotenv	Lectura de variables de entorno y

5. Requisitos de software

- Node.js LTS instalado.
- npm disponible.
- MySQL Server instalado y activo.
- MySQL Workbench opcional para administración visual.
- Git instalado para control de versiones.
- Navegador web moderno.
- Editor de código, por ejemplo Visual Studio Code.
- Cuenta y credenciales para el servicio de IA seleccionado.

6. Requisitos de hardware sugeridos

Recurso	Mínimo sugerido
Procesador	2 núcleos o superior
Memoria RAM	8 GB
Almacenamiento	5 GB libres para entorno, dependencias y documentos de prueba
Red	Conectividad a internet si se consume una API externa de IA

7. Estructura del proyecto

```
documind-ai/
├── config/
├── controllers/
├── database/
├── middleware/
├── models/
├── public/
│   ├── css/
│   └── js/
├── routes/
├── services/
│   ├── documentService.js
│   ├── aiService.js
│   └── searchService.js
├── uploads/
├── views/
├── .env
├── .gitignore
├── app.js
└── package.json
```

8. Configuración de variables de entorno

Las credenciales y secretos no deben escribirse directamente en el código ni publicarse en GitHub. Se propone un archivo `.env` local y un archivo `.env.example` sin secretos.

```
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=CAMBIAR
DB_NAME=documind_ai
JWT_SECRET=CAMBIAR
AI_API_KEY=CAMBIAR
UPLOAD_DIR=uploads
```

El archivo .gitignore debe incluir .env, node_modules y cualquier archivo temporal o secreto.

9. Configuración de base de datos

1. Crear la base de datos documind_ai.
2. Ejecutar el script SQL incluido en la carpeta 10-base-de-datos o database.
3. Verificar la creación de tablas y relaciones.
4. Configurar usuario, contraseña, host y puerto en .env.
5. Probar la conexión desde el backend antes de iniciar pruebas funcionales.

10. Modelo de datos técnico

Entidad	Propósito
usuarios	Credenciales, rol y estado de acceso.
repositorios	Carpetas lógicas creadas por los usuarios.
documentos	Metadatos de archivos cargados, ruta, formato y estado.
procesamientos	Resultados de clasificación, resumen y extracción.
documento_chunks	Fragmentos del contenido usados para búsqueda o RAG.
consultas	Preguntas realizadas y respuestas generadas.
errores	Registro técnico de fallos de extracción, IA u otros procesos.

11. Flujo técnico de carga y procesamiento

1. El usuario autenticado selecciona un repositorio y carga un archivo.
2. Multer recibe el archivo y valida extensión/tamaño.
3. El backend registra el documento con estado Pendiente.
4. El servicio documental extrae texto según PDF, DOCX o TXT.
5. El contenido extraído se normaliza y se envía al servicio de IA.
6. La IA devuelve categoría, resumen y campos extraídos.
7. Los resultados se almacenan en la base de datos.

- 8. El texto puede dividirse en fragmentos para búsqueda y consulta.
- 9. El documento cambia a estado Procesado; ante fallos, se registra Error.

12. Integración con inteligencia artificial

El módulo aiService.js debe encapsular la comunicación con el proveedor de IA. Esto evita acoplar las rutas y controladores a una API específica y facilita cambiar de proveedor.

- Entrada: texto extraído y tipo de operación.
- Operaciones: clasificación, resumen, extracción estructurada y consulta en lenguaje natural.
- Salida: JSON validado por el backend cuando la operación requiera datos estructurados.
- Errores: timeouts, respuestas inválidas y límites de API deben capturarse y registrarse.
- Privacidad: se debe evitar enviar información innecesaria y utilizar únicamente documentos de prueba o datos autorizados.

13. Búsqueda y consulta documental

La aplicación debe permitir búsqueda dentro del contenido. Para la consulta en lenguaje natural se propone un flujo RAG sencillo: recuperar fragmentos relevantes del repositorio y suministrarlos como contexto al modelo antes de generar la respuesta.

Pregunta -> Recuperación de fragmentos -> Contexto -> IA -> Respuesta

14. Endpoints principales propuestos

Método	Ruta	Uso
POST	/api/auth/login	Autenticación.
POST	/api/repositorios	Crear repositorio.
GET	/api/repositorios	Listar repositorios autorizados.
POST	/api/documentos	Cargar documento.
GET	/api/documentos/:id	Consultar detalle y resultados.
GET	/api/documentos/:id/download	Descargar archivo.
DELETE	/api/documentos/:id	Eliminar archivo y datos asociados.
GET	/api/busqueda	Buscar contenido documental.
POST	/api/consultas	Realizar pregunta en lenguaje natural.
GET	/api/dashboard	Obtener indicadores.
GET	/api/admin/errores	Consultar errores de procesamiento.

15. Seguridad básica

- Almacenar contraseñas con bcrypt; nunca en texto plano.
- Usar JWT o sesión para proteger rutas privadas.
- Validar tipo MIME, extensión y tamaño de archivos.
- Sanitizar y validar entradas del usuario.
- Usar consultas parametrizadas en MySQL.
- No exponer API Keys ni secretos en frontend o Git.
- Restringir acceso a archivos según usuario/repositorio.
- Registrar fallos sin mostrar detalles internos sensibles al usuario final.

16. Instalación local

1. Clonar el repositorio Git.
2. Abrir una terminal en la carpeta del código fuente.
3. Ejecutar npm install.
4. Crear .env a partir de .env.example.
5. Configurar MySQL y ejecutar scripts SQL.
6. Crear la carpeta uploads si no existe.
7. Configurar la API Key de IA.
8. Ejecutar npm start o el script definido en package.json.
9. Abrir http://localhost:3000 o el puerto configurado.

17. Administración del sistema

- Revisar usuarios activos y roles.
- Supervisar documentos pendientes, procesados y con error.
- Revisar el registro de errores y reintentar procesos cuando sea posible.
- Controlar crecimiento del almacenamiento local.
- Revisar consumo y límites del proveedor de IA.
- Mantener respaldos de base de datos y documentos cargados.

18. Registro y manejo de errores

Origen	Ejemplo	Acción
Carga	Formato o tamaño inválido	Rechazar el archivo y devolver mensaje controlado.
Extracción	PDF sin texto legible	Registrar error o marcar posible necesidad de OCR.
IA	Timeout o cuota agotada	Registrar fallo, conservar documento y permitir reintento.
Base de datos	Conexión no disponible	Registrar error técnico y evitar pérdida silenciosa.
Búsqueda	Índice o fragmentos no disponibles	Informar que el documento aún no está listo para consulta.

19. Respaldo y recuperación

- Realizar copia periódica de la base MySQL.
- Respaldar la carpeta de documentos cargados.
- Mantener scripts de creación de esquema versionados en Git.
- No incluir secretos en copias compartidas.
- Probar periódicamente una restauración en un ambiente separado.

20. Mantenimiento

Frecuencia	Actividad
Semanal	Revisar errores de procesamiento y espacio

	disponible.
Mensual	Respalidar base de datos y documentos; revisar dependencias y consumo de IA.
Por versión	Ejecutar pruebas de regresión y actualizar documentación.
Cuando aplique	Rotar secretos, actualizar paquetes vulnerables y revisar permisos.

21. Control de versiones con Git

- Realizar commits descriptivos y pequeños.
- No subir .env ni API Keys.
- Mantener README actualizado.
- Usar ramas para cambios relevantes cuando el equipo trabaje en paralelo.
- Etiquetar o documentar versiones entregadas para permitir reproducibilidad.

22. Diagnóstico rápido de incidencias

Síntoma	Verificación inicial
La aplicación no inicia	Revisar Node.js, npm install, variables de entorno y puerto.
No conecta a MySQL	Validar servicio MySQL, usuario, contraseña, host, puerto y nombre de base.
No carga archivos	Revisar permisos de uploads, Multer, tamaño y formato.
No procesa documentos	Revisar extracción de texto, estado del servicio IA y logs.
No responde preguntas	Confirmar que el documento esté procesado e indexado y que la API de IA esté disponible.
Dashboard vacío	Verificar que existan datos en la base y que el endpoint responda correctamente.

23. Evidencias técnicas pendientes

Las capturas de terminal, consultas SQL, estructura real del repositorio, configuración del servicio de IA y evidencias de despliegue se incorporarán cuando la aplicación esté implementada. La guía académica exige que dichas evidencias correspondan a ejecuciones reales.

24. Conclusión

Este manual establece una referencia técnica reproducible para DocuMind AI. La separación entre frontend, backend, datos, almacenamiento e IA facilita la administración y evolución del sistema y permite documentar de manera clara el entorno requerido para su ejecución y soporte.